

Promptathon — ICEPOT Prompt Makeover Challenge

Scenarion-1: Role-Based Access Control of IRCTC App

LLM Model used: **Gemini**

Weak prompt: Write test cases for user roles.

Output:

To write effective test cases for user roles (Role-Based Access Control, or RBAC), you need to verify two main things: **authentication** (can they log in?) and **authorization** (can they only do what they are supposed to do?).

Here is a comprehensive framework and template for testing user roles, using a standard **Admin, Manager, and Viewer** hierarchy as an example.

1. Test Case Matrix (Quick Reference)

Before writing detailed steps, a matrix helps you visualize who should have access to what.

User Role	Create/Delete Users	Edit Data	View Reports	Access Billing
Admin	✔ Yes	✔ Yes	✔ Yes	✔ Yes
Manager	✘ No	✔ Yes	✔ Yes	✘ No
Viewer	✘ No	✘ No	✔ Yes	✘ No

2. Detailed Test Cases

Category A: Positive Testing (Intended Access)

These ensure that roles can successfully perform their designated duties.

Test Case 1: Admin Full Access Verification

- **Description:** Verify that the Admin user can access all sections and perform all CRUD (Create, Read, Update, Delete) operations.
- **Pre-conditions:** Admin user account exists and is active.

- **Test Steps:**
 1. Log in as the Admin user.
 2. Navigate to the User Management dashboard and delete a test user.
 3. Navigate to the Billing section.
 4. Modify system settings.
- **Expected Result:** All actions are successful; no "Access Denied" errors occur.

Test Case 2: Manager Restrictive Write Access

- **Description:** Verify that a Manager can edit data and view reports but cannot touch user management.
- **Pre-conditions:** Manager user account exists.
- **Test Steps:**
 1. Log in as the Manager user.
 2. Navigate to the Data/Projects dashboard and click "Edit" on an entry.
 3. Save the changes.
- **Expected Result:** Changes are saved successfully. The User Management and Billing tabs should either be hidden or explicitly disabled.

Category B: Negative Testing (Unauthorized Access)

These are crucial for security. They ensure users *cannot* bypass restrictions, even if they try to trick the system.

Test Case 3: Viewer Privilege Escalation (UI Level)

- **Description:** Verify that a Viewer cannot see or click modification buttons.
- **Pre-conditions:** Viewer user account exists.
- **Test Steps:**
 1. Log in as the Viewer user.
 2. Navigate to the main dashboard.
 3. Look for "Delete," "Edit," or "Invite User" buttons.
- **Expected Result:** Edit/Delete buttons are completely hidden or greyed out. Clicking them (if visible) does nothing.

Test Case 4: URL/API Hacking (Crucial Security Test)

- **Description:** Verify that a lower-role user cannot access a restricted page by directly typing the URL or hitting the API endpoint.
- **Pre-conditions:** Viewer user is logged in. The Admin URL path is known (e.g., /admin/settings).
- **Test Steps:**
 1. Log in as a Viewer.
 2. Manually type the Admin URL (https://app.com/admin/settings) into the browser address bar and press Enter.
 3. (Optional API Test): Attempt to send a POST request to /api/v1/users using the Viewer's auth token.
- **Expected Result:** The system redirects the user to a **403 Forbidden** or **404 Not Found** page. The API request returns a 403 Unauthorized status code.

Category C: Boundary & Lifecycle Testing

Test Case 5: Dynamic Role Changing (Session Update)

- **Description:** Verify that a user's permissions update correctly when an Admin changes their role mid-session.
- **Test Steps:**
 1. User A logs in as a **Viewer**.
 2. In a separate browser, the Admin changes User A's role to **Manager**.
 3. User A refreshes their page or attempts to perform a Manager action (e.g., editing data).
- **Expected Result:** User A instantly gains Manager privileges (or is forced to re-authenticate), and can now edit data without logging out and back in.

Test Case 6: Multiple Roles Conflicts

- **Description:** If the system allows a user to have multiple roles (e.g., "Billing Specialist" AND "Viewer"), verify how permissions inherit.
- **Expected Result:** The user should receive the *highest* cumulative permissions of both roles, unless the business logic dictates otherwise.



Pro-Tips for Automation

Don't hardcode credentials: Use environment variables for your test users (e.g., `process.env.ADMIN_USER`).

Test the API first: Testing roles via UI is slow. Write automated API integration tests checking for 403 Forbidden statuses for restricted roles—it will save you hours of testing time.

Strong Prompt:

Context:

Act as a Senior Software Test Engineer responsible for designing comprehensive Role-Based Access Control (RBAC) test cases for the IRCTC Train Booking application (<https://www.irctc.co.in/>).

The application supports the following user roles:

- Guest
- Passenger
- Agent

- Support
- Admin

The application enforces Role-Based Access Control (RBAC), where each role has specific permissions and restrictions based on the following business rules.

Business Rules:

Guest

- Can search trains, view train schedules, fares, and seat availability without logging in.
- Can check PNR status without authentication.
- Cannot book tickets, cancel tickets, make payments, manage passengers, or access booking history.
- Must be redirected to the Login page when attempting to access protected features.

Passenger

- Can search trains, book tickets, cancel their own tickets, make payments, and manage their own passenger details.
- Can view only their own booking history and profile information.
- Can request refunds only for their own confirmed tickets.
- Cannot approve or process refund requests.
- Cannot access or modify another passenger's data.

Agent

- Can search trains and book tickets on behalf of customers.
- Can cancel tickets booked through their own agent account.
- Can manage passenger information related to their bookings.
- Can view booking history and reports associated with their agent account.
- Cannot access or modify another agent's bookings or customer data.

- Cannot perform administrative functions such as user management or application configuration.

Support

- Can view customer booking details to assist with support requests.
- Can cancel tickets only after successful customer verification.
- Cannot initiate or book new tickets.
- Cannot modify payment information or passenger profile details.
- Can assist users only within the scope of support-related activities.

Admin

- Can manage users, roles, permissions, and application configurations.
- Can view payment records and configure payment settings.
- Can access reports, audit logs, and system administration features.
- Cannot perform payment transactions on behalf of customers.
- Must ensure all administrative actions are recorded in audit logs.

Assume that all role-based permissions must be enforced consistently across the User Interface (UI), APIs, backend services, and direct URL access.

Instruction:

[MANDATORY] Generate comprehensive test cases for role based access control of the IRCTC Train Booking App where in the available roles are as follows:

[MANDATORY] Cover both positive and negative scenarios for each user role.

[MANDATORY] Include UI validation, API authorization, backend validation, session management, direct URL access, privilege escalation, and security-related test cases.

[MANDATORY] Ensure that the test cases are suitable for enterprise-level QA and provide complete functional coverage.

Example:

Example 1: Guest User – Book Ticket (Negative)

Test Case ID

RBAC_BOOK_001

Test Case Title

Verify that a Guest User is not allowed to book a train ticket.

Objective

To verify that an unauthenticated (Guest) user cannot access the ticket booking functionality and is prompted to log in before proceeding.

Pre-requisites

User is not logged into the application.

Train search page is accessible.

Test Steps

Launch the IRCTC application.

Search for trains between Chennai and Bangalore.

Select an available train.

Click on the Book Now button.

Observe the application behavior.

Expected Result

The application should redirect the user to the Login page.

The booking page should not be displayed.

No booking should be initiated.

An appropriate message such as "Please login to continue booking" should be displayed.

The application should not expose any booking-related APIs or customer information.

Example 2: Passenger – Book Ticket (Positive)

Test Case ID

RBAC_BOOK_002

Test Case Title

Verify that a Registered Passenger can successfully book a train ticket.

Objective

To verify that a registered passenger has the necessary permissions to access the booking functionality and successfully complete a ticket reservation.

Pre-requisites

Passenger account is active.

Passenger is logged into the application.

Seats are available for the selected train.

Test Steps

Login using a valid Passenger account.

Search for trains between Chennai and Bangalore.

Select an available train.

Enter passenger details.

Proceed to payment.

Complete the payment successfully.

Expected Result

The booking should be completed successfully.

A unique Booking ID/PNR should be generated.

The booked ticket should appear in the Booking History.

A confirmation notification or email should be sent to the passenger.

Persona:

Act as:

- Senior QA Architect
- Security Test Engineer
- Automation Test Lead
- Product Owner
- Business Analyst
- End user

While generating test cases, think from all these perspectives simultaneously.

Identify hidden security risks, missing authorization checks, privilege escalation opportunities, business rule violations, usability issues, and automation feasibility.

Suggest additional test scenarios whenever you identify coverage gaps.

Output:

Generate the output in a downloadable .xlsx format

Tone:

Professional Tone with clarity

Output:

Your XLSX file is ready



IRCTC_RBAC...Test_Suite

XLSX

Summary of the Generated Enterprise-Grade RBAC Test Suite

As a Senior QA Architect, Security Engineer, and Automation Test Lead, I have designed a comprehensive, multi-layered Role-Based Access Control (RBAC) test suite for the IRCTC Train Booking Application. The test cases have been meticulously built and exported into a professional, presentation-ready Excel workbook (.xlsx) adhering to standard enterprise conventions.

The spreadsheet is divided into three functional tabs:

- 1. Dashboard:** Contains high-level project telemetry, overview, and formula-driven metrics tracking the breakdown of test case coverage.
- 2. RBAC Matrix:** A clear, visually structured entitlements boundary table mapping all 5 user roles (Guest, Passenger, Agent, Support, Admin) across core system components.
- 3. Test Cases:** An enterprise-grade, deeply descriptive test case repository that goes significantly beyond surface-level steps to enforce security, backend assertions, and data validation layers.

Key Architectural Testing Pillars Covered

To ensure complete production-level coverage, each scenario has been analyzed and evaluated from multiple engineering angles:

- Multi-Layer Verification (Defense-in-Depth):** Test steps and expected results specify validation across the User Interface (Route Guards, hidden elements, error alerts), API Authorization (Gateway checks, HTTP response codes like 401 Unauthorized/403 Forbidden), and Backend/DB Integrity (Data isolation and transaction rollbacks).
- Security Vulnerability Mapping:** Heavy emphasis is placed on identifying critical OWASP Top 10 API Security Risks, such as:
 - BOLA / IDOR (Broken Object Level Authorization):** Verifying that a Passenger or Agent cannot alter or view cross-tenant booking history by manipulating PNRs or User IDs in the request payload.
 - BFLA (Broken Function Level Authorization):** Validating that backend administrative microservices reject requests from lower-privileged roles like Passengers or Agents, even if they attempt direct URL injection or payload modification.
 - JWT Token Tampering:** Verifying that backend microservices immediately invalidate and reject tokens if signature strings or structural properties are altered via proxies.
- Automation Feasibility & Strategy:** Every single test case includes a dedicated column defining exact strategies for automated test script development (e.g.,

using *Selenium/Cypress* for UI flows, *RestAssured/PyTest* for contract-level API assertions, and handling token injections).

- **Architect & Hidden Risk Notes:** Highlights edge cases such as concurrency bottlenecks during high-traffic booking cancellations, data sanitization to prevent SQL injection, and PII masking inside public PNR inquiry payload data.

Overview of Included Test Cases

The test suite covers both Positive (Permitted Actions) and Negative (Restricted & Misuse Actions) boundaries for every role, including:

1. **Guest User:** Public train scheduling (Positive), unauthenticated PNR tracking with data masking (Positive), direct URL deep-link traversal blocks (Negative), and unauthenticated API request rejections (Negative).
2. **Passenger:** Complete booking lifecycle (Positive), multi-tenant historical data isolation / IDOR validation (Negative), route-guard privilege escalation mitigation (Negative), and isolated cancellation and refund handling (Positive/Negative).
3. **Agent:** Dedicated B2B commercial channel processing (Positive), cross-agent data isolation boundaries (Negative), administrative endpoint lockouts (Negative), and agent wallet transaction ledger checking (Positive).
4. **Support User:** Read-only triage query interfaces (Positive), system-wide booking creation blocks (Negative), profile/financial mutation constraint verification (Negative), and verified customer cancellation workflows (Positive).
5. **Admin User:** Global system rule configurations via step-up authentication (Positive), passenger booking engine channel isolation (Negative), multi-tenant user permission assignment propagation (Positive), and immutable, non-repudiation system security audit logging (Positive).

Professional Formatting Standards Applied

- **Color-Coded Status Matching:** Subtle, desaturated light green (#E2EFDA) blocks indicate positive scenario verification patterns, while soft orange-red (#FCE4D6) denotes negative security/privilege separation checks.
- **Formula-Driven Integrity:** The Summary metrics page uses native uppercase Excel formulas (COUNTA, COUNTIF) dynamically tied to the live row ranges in the Test Cases sheet.
- **Ready for Presentation:** Structured with fixed column widths, text wrap constraints, bold headers, gridlines enabled, and typography designed for clear visual reading.

Observations on the improvements

1. The weak prompt produced generic test cases that did not validate role-specific permissions and functionalities for each user role.
2. The generated test cases followed a generic template instead of the required test case format and fields.

3. The user roles used in the generated test cases were generic and were not aligned with the actual roles defined in the IRCTC application
4. Received a comprehensive list of test cases in a well-structured, Excel-friendly format with excellent coverage of functional, business, and edge-case scenarios.

Scenarion-2: *Shopping Cart (Flipkart)*

LLM Model Used: **claude.ai**

Weak Prompt: Test the cart functionality

Output:

I don't have access to any cart or application to test — no URL, codebase, or file has been shared in this conversation.

Could you share what you'd like me to test? For example:

- A URL to a website with a cart
- Code files for a cart implementation
- A specific cart feature or scenario you want tested

Strong Prompt:

Instructions:

Act as a ****Senior Software Test Engineer with 12+ years of experience in e-commerce application testing****.

Generate ****comprehensive positive, negative, boundary, and edge-case test cases**** for the ****Shopping Cart**** feature of the Flipkart mobile application.

The test cases should cover both ****functional**** and ****business rule**** validations. Ensure no critical shopping cart functionality is missed.

Context:

The Shopping Cart allows users to:

* Add products to the cart

- * Remove products from the cart
- * Update item quantities
- * Move items to the Wishlist
- * View product details in the cart
- * Apply and remove coupons or promo codes
- * View discounts and offers
- * View the order summary
- * Proceed to checkout

The following business rules must be validated:

Cart Operations

- * Users can add one or multiple products to the cart.
- * Users can increase or decrease the quantity within the allowed purchase limit.
- * Quantity cannot exceed available inventory.
- * Users can remove individual products.
- * Users can empty the entire cart.
- * Users can move products between the Cart and Wishlist.

Price Calculation

Validate the correctness of:

- * Product subtotal
- * Quantity-wise total
- * Delivery charges
- * Platform fee (if applicable)
- * Taxes (if applicable)

- * Packaging charges (if applicable)
- * Discounts
- * Final payable amount

Ensure all totals are recalculated correctly whenever:

- * Quantity changes
- * Items are added
- * Items are removed
- * Coupons are applied or removed

Coupons & Discounts

Include scenarios for:

- * Valid coupon application
- * Invalid coupon
- * Expired coupon
- * Coupon minimum order value validation
- * Coupon usage limit exceeded
- * Coupon applicable only to selected categories or products
- * One-time-use coupons
- * Stackable vs non-stackable coupons
- * Automatic promotional discounts
- * Removal of applied coupons

Inventory & Stock Handling

Validate:

- * Out-of-stock products already present in the cart
- * Product becoming out of stock after being added
- * Limited stock warnings
- * Quantity exceeding available stock
- * Product discontinued after being added
- * Inventory updates during checkout

Business Rules

Validate:

- * Maximum purchase quantity
- * Duplicate product handling
- * Guest user vs logged-in user cart behavior
- * Cart persistence after logout/login
- * Cart synchronization across multiple devices
- * Saved-for-later functionality
- * Session timeout handling
- * Network interruption during cart updates

Error Handling

Validate:

- * API failures
- * Slow network
- * Server timeout
- * Invalid product ID

- * Price changes after adding to the cart
- * Product unavailable during checkout

Example

Generate test cases using the following format:

Test ID	Test Scenario	Pre-Conditions	Test Steps
Test Data	Expected Result		Priority
-----	-----	-----	-----
-----	-----	-----	-----
-----	-----	-----	-----
CART-001	Verify user can add a product to the cart	User is logged in	1. Search for a product 2. Open product page 3. Tap **Add to Cart** Product: iPhone 16 Product is successfully added to the cart and the cart count is updated High

Persona:

Assume the roles of:

- * Senior Software Test Engineer
- * QA Test Lead
- * E-commerce Domain Expert
- * Business Analyst
- * Product Owner

Review the generated test cases from each perspective to ensure:

- * Complete functional coverage
- * Validation of business rules
- * Edge-case identification
- * Risk-based testing
- * User experience considerations

Output:

Generate the output in a **Markdown table** with the following columns:

- * Test ID
- * Module
- * Test Scenario
- * Test Type (Positive/Negative/Boundary/Edge)
- * Pre-Conditions
- * Test Steps
- * Test Data
- * Expected Result
- * Priority (High/Medium/Low)

Additionally:

1. Group test cases by feature:

- * Add to Cart
- * Remove from Cart
- * Update Quantity
- * Price Calculation

- * Coupons & Discounts
- * Inventory Validation
- * Checkout Readiness
- * Error Handling

2. Include:

- * Positive scenarios
- * Negative scenarios
- * Boundary value scenarios
- * Edge cases

3. Highlight any assumptions made where Flipkart-specific business rules are not publicly available.

Tone:

Use a ****professional QA documentation style**** suitable for:

- * Test planning
- * Manual testing
- * Automation scripting
- * QA reviews
- * Client documentation

The test cases should be detailed, unambiguous, and ready for execution by both manual and automation testers.

Output:

Check frontend design skill for artifact styling

Check frontend design skill for artifact styling

Now let me generate the comprehensive test cases and create a well-formatted artifact.

Created a file, read a file

Created a file, read a file

Here's your comprehensive test case document — **84 test cases** across all 8 functional groups.
Here's a quick summary of what was produced:

What's Covered

Group	TCs	Highlights
Add to Cart	13	Guest/logged-in, variant selection, device sync, network interruption
Remove from Cart	7	Undo, network edge cases, empty cart state
Update Quantity	10	Boundary values (0, 1, max, max+1), race conditions, coupon re-validation
Price Calculation	12	Multi-product totals, delivery thresholds, tax lines, price change alerts
Coupons & Discounts	15	Invalid/expired/OOL coupons, stackable, auto-promo, cap limits
Inventory Validation	7	OOS detection, real-time sync, concurrent buyer conflict
Checkout Readiness	10	Guest-to-login flow, cart persistence, Wishlist/Saved for Later
Error Handling	10	API failures, timeout, offline mode, price-sync failure

Observations on the improvements:

1. When used the weak prompt, the AI could not even understand, it was asked to generate the test cases, it simply replied like, its not having access to the application for testing
2. Got good quality test cases with the precise meaning and got them in very structure manner
3. Got the test cases specific to the Application under test

Scenarion-2: *File Upload (Canva)*

Model: Qwen

Weak prompt: Generate test cases for file upload.

Output: Here is a comprehensive set of test cases for a **File Upload** feature, categorized by testing type. You can adapt the specific file types, size limits, and UI elements to match your application's exact requirements.

1. Functional Test Cases (Positive)

ID	Test Scenario	Steps	Expected Result
F01	Upload a valid single file within size limit	1. Click "Upload". 2. Select a valid file (e.g., document.pdf, 2MB). 3. Click "Submit".	File uploads successfully. Success message is displayed. File is saved correctly on the server.
F02	Upload multiple valid files simultaneously	1. Click "Upload". 2. Select multiple valid files (e.g., 3 images). 3. Click "Submit".	All files upload successfully. Success message shows the count of uploaded files.
F03	Drag and drop a valid file	1. Drag a valid file from the OS file explorer. 2. Drop it into the designated upload zone.	File is accepted, upload starts automatically, and a success message is shown.
F04	Upload a file with spaces and special characters in the name	1. Select a file named my report (final) v2!.pdf. 2. Upload.	File uploads successfully. The system safely handles or sanitizes the filename without errors.

F05	Cancel an ongoing upload	1. Start uploading a moderately large file. 2. Click the "Cancel" or "X" button during upload.	Upload stops immediately. Temporary files are cleaned up. UI reverts to the initial state.
------------	--------------------------	---	--

2. Functional Test Cases (Negative)

ID	Test Scenario	Steps	Expected Result
F06	Upload a file exceeding the maximum size limit	1. Select a file larger than the allowed limit (e.g., 55MB if limit is 50MB). 2. Attempt to upload.	System prevents the upload. A clear, user-friendly error message is displayed (e.g., "File exceeds 50MB limit").
F07	Upload an unsupported file type	1. Select a file with an invalid extension (e.g., .exe, .bat when only .pdf and .jpg are allowed). 2. Attempt to upload.	System rejects the file. Error message specifies allowed file types.
F08	Upload a zero-byte (empty) file	1. Create an empty file (0 bytes). 2. Attempt to upload.	System rejects the file with an appropriate error message (e.g., "File is empty").
F09	Attempt upload without selecting a file	1. Leave the file input empty. 2. Click "Submit".	System prevents submission and shows a validation error (e.g., "Please select a file").
F10	Upload a file with a very long filename	1. Select a file with a filename exceeding 255 characters. 2. Attempt to upload.	System handles it gracefully (either truncates safely or rejects with a clear error), preventing server-side crashes.

3. Security Test Cases

ID	Test Scenario	Steps	Expected Result
S01	MIME Type Spoofing	1. Rename a malicious .exe file to .pdf. 2. Attempt to upload.	Server validates the actual file content (magic numbers), not just the extension, and rejects the file.
S02	Double Extension Attack	1. Name a file malware.php.jpg or script.js.pdf. 2. Attempt to upload.	Server rejects the file or strips the dangerous extension, ensuring it cannot be executed.

S03	Path Traversal in Filename	1. Intercept the request and change the filename to ../.././etc/passwd or ../../windows/system32. 2. Submit.	Server sanitizes the filename, ignoring directory traversal sequences, and saves it in the designated upload folder only.
S04	XSS via Filename	1. Name a file <code><script>alert(1)</script>.jpg</code> . 2. Upload and view the file in the UI.	The filename is properly HTML-encoded when displayed, preventing script execution.
S05	Upload of Giant File (DoS Attack)	1. Attempt to upload a massively large file (e.g., 10GB) via API manipulation (bypassing UI limits).	Server enforces strict size limits at the web server/proxy level (e.g., Nginx <code>client_max_body_size</code>) and drops the connection with a 413 Payload Too Large error.
S06	Concurrent Upload Limit Exhaustion	1. Use a script to initiate hundreds of simultaneous upload requests.	Rate limiting is triggered. Subsequent requests are blocked (e.g., 429 Too Many Requests).

4. Performance & Load Test Cases

ID	Test Scenario	Steps	Expected Result
P01	Upload large files under normal network conditions	1. Upload a file near the max limit (e.g., 50MB).	Upload completes within an acceptable timeframe. Progress bar updates smoothly.
P02	Upload under throttled network conditions	1. Use browser DevTools to throttle network to "Slow 3G". 2. Upload a 10MB file.	Upload continues without crashing. Timeout limits are reasonable. Resumable upload (if supported) works.
P03	High concurrency load testing	1. Simulate 100+ users uploading files simultaneously using a tool like JMeter or k6.	Server CPU/Memory remains stable. No database deadlocks or file system locks occur.

5. Usability & UI/UX Test Cases

ID	Test Scenario	Steps	Expected Result
U01	Verify Upload Progress Indicator	1. Upload a 20MB file. 2. Observe the UI.	A progress bar or spinner is visible, accurately reflecting the upload percentage.

U02	Verify clear error messaging	1. Trigger an error (e.g., wrong file type).	Error message is specific, helpful, and visually distinct (e.g., red text), not a generic "Error 500".
U03	Keyboard accessibility	1. Use Tab to navigate to the upload button. 2. Press Enter or Space to open the file dialog.	File dialog opens. The feature is fully accessible without a mouse.
U04	Drag and drop visual feedback	1. Drag a file over the upload zone.	The upload zone changes appearance (e.g., border highlights, background color changes) to indicate it is an active drop target.

6. Compatibility Test Cases

ID	Test Scenario	Steps	Expected Result
C01	Cross-browser compatibility	1. Test the upload feature on Chrome, Firefox, Safari, and Edge.	Feature works identically across all major modern browsers.
C02	Cross-platform/mobile compatibility	1. Test on iOS (Safari), Android (Chrome), Windows, and macOS.	File picker opens correctly (e.g., offers camera/gallery options on mobile). Upload succeeds.
C03	Different file systems	1. Upload files originating from NTFS (Windows), APFS (macOS), and ext4 (Linux).	Filenames and metadata are handled correctly without encoding issues.

7. Edge Cases

ID	Test Scenario	Steps	Expected Result
E01	Network interruption during upload	1. Start an upload. 2. Disconnect Wi-Fi or turn on Airplane mode. 3. Reconnect.	System either fails gracefully with a "Network error, please retry" message or automatically resumes the upload (if chunked/resumable upload is implemented).

E02	Uploading a file currently being written to	1. Start downloading or copying a large file on the OS. 2. Immediately select and upload that same file before the OS finishes writing it.	System handles the file lock gracefully or waits, rather than crashing or uploading a corrupted partial file.
E03	Duplicate file upload	1. Upload report.pdf. 2. Immediately upload report.pdf again.	System handles it based on business logic: either overwrites, appends a suffix (e.g., report(1).pdf), or rejects with a "File already exists" message.



Best Practices for File Upload Testing

1. **Always validate on the server:** Never trust client-side validation (JS/HTML). Attackers can easily bypass it.
2. **Use Chunked Uploads:** For files >10MB, implement chunked/resumable uploads to improve reliability and UX.
3. **Scan for Malware:** Integrate server-side antivirus scanning (e.g., ClamAV) for uploaded files if they will be downloaded by other users.
4. **Store Safely:** Store uploaded files outside the web root directory, or serve them via a secure CDN with strict Content-Type headers and Content-Disposition: attachment.
5. **Automation Tools:** Use **Selenium/Playwright** for UI upload testing, **Postman/RestAssured** for API upload testing, and **OWASP ZAP** for security validation.

Strong Prompt:

Instructions:

Act as a **Senior Software Test Engineer with over 12 years of experience in testing web and mobile applications.**

Generate **comprehensive positive, negative, boundary, edge-case, usability, and security test cases for the **File Upload** functionality of the Canva application.**

Ensure the test cases validate both **functional behavior and **non-functional aspects**, including security, performance, compatibility, and user experience.**

Context:

The Canva application allows users to upload files that can be used in their designs. Users can upload files from local storage, cloud storage, or mobile devices.

The upload feature supports various file types and validates uploaded content before making it available in the user's media library.

Supported File Types

Generate test cases for validating uploads of:

****Images****

- * JPG**
- * JPEG**
- * PNG**
- * SVG**
- * GIF**
- * WEBP**
- * BMP**
- * TIFF**

****Documents****

- * PDF**

****Videos****

- * MP4
- * MOV
- * AVI
- * WEBM

****Audio****

- * MP3
- * WAV

Include scenarios for:

- * Supported formats
- * Unsupported formats
- * Corrupted files
- * Password-protected files (where applicable)

File Size Validation

Validate uploads for:

- * Minimum file size
- * Maximum supported file size
- * Zero-byte files
- * Files exactly at the size limit
- * Files just below the limit
- * Files just above the limit

- * Extremely large files
- * Multiple files whose combined size exceeds the limit

Upload Operations

Generate test cases for:

- * Single file upload
- * Multiple file upload
- * Drag-and-drop upload
- * Upload using file browser
- * Retry failed upload
- * Cancel upload
- * Resume interrupted upload (if supported)
- * Replace existing uploaded file
- * Duplicate file upload
- * Upload progress indicator
- * Simultaneous uploads

Success Criteria

Validate that:

- * Upload completes successfully
- * Success message is displayed

- * Uploaded file appears in the user's media library
- * File preview is generated correctly
- * Metadata (file name, size, type) is displayed accurately
- * Uploaded asset can be inserted into a design
- * Upload time is within acceptable limits

Failure Criteria

Validate scenarios where:

- * Network connection is lost
- * Server returns an error
- * Upload times out
- * Unsupported file format is uploaded
- * File exceeds the maximum size limit
- * File is corrupted
- * Storage quota is exceeded
- * Duplicate upload is not allowed (if applicable)

Ensure the application displays meaningful error messages and allows recovery where appropriate.

Security Validation

Generate security-focused test cases for:

- * Malicious executable files (.exe, .bat, .cmd, .sh)
- * Files containing embedded malware
- * Script injection through file names
- * HTML or JavaScript files disguised as images
- * Double-extension files (example: image.jpg.exe)
- * Incorrect MIME type validation
- * Virus-infected files
- * Oversized files intended to exhaust server resources
- * Directory traversal attempts in file names
- * Special characters and Unicode characters in file names
- * SQL Injection strings in file names
- * Cross-Site Scripting (XSS) payloads in file names or metadata
- * Compressed archive files containing malicious content (ZIP bombs)
- * Unauthorized users attempting uploads
- * Uploads over insecure connections (where applicable)

Verify that the application safely rejects malicious files and does not expose sensitive system information in error messages.

Performance & Reliability

Generate test cases for:

- * Upload performance under slow network conditions
- * Upload performance on fast networks
- * Large batch uploads

- * Concurrent uploads by multiple users
- * Memory and CPU utilization during uploads
- * Recovery after application crash
- * Upload interruption due to network switching
- * Mobile background/foreground upload behavior (if supported)

Compatibility Testing

Validate uploads from:

- * Chrome
- * Firefox
- * Edge
- * Safari

Also include:

- * Windows
- * macOS
- * Android
- * iOS

Verify behavior across different screen sizes and devices.

Example:

Generate test cases using the following template:

- | Test ID | Module | Test Scenario | Test Type | Pre-Conditions | Test Steps | Test Data | Expected Result |
|---------|-------------|--|-----------|-------------------|---|------------------|---|
| UPL-001 | File Upload | Verify a user can upload a valid PNG image | Positive | User is logged in | 1. Open Upload panel
2. Select a PNG file
3. Click Upload | image.png (2 MB) | Upload completes successfully, success message is displayed, and the image appears in the media library |
| | | | | | | | High |

- * Excellent user experience
- * Risk-based testing

Output:

Generate the test cases in a **Markdown table**** with the following columns:**

- * Test ID
- * Module
- * Test Scenario
- * Test Type (Positive / Negative / Boundary / Edge / Security / Performance / Compatibility)
- * Pre-Conditions
- * Test Steps
- * Test Data
- * Expected Result
- * Priority

Group the test cases into the following sections:

1. Supported File Types
2. Unsupported File Types
3. File Size Validation
4. Upload Operations
5. Success Scenarios
6. Failure Scenarios
7. Security Testing

8. Performance Testing

9. Compatibility Testing

10. Edge Cases

Highlight any assumptions made where Canva-specific implementation details are not publicly available.

Tone:

Use a *professional QA documentation style*** suitable for:**

*** Manual testing**

*** Automation test design**

*** QA reviews**

*** Test planning**

*** Client documentation**

The generated test cases should be detailed, unambiguous, reusable, and ready for execution by both manual and automation testers.

Output:

Comprehensive Test Cases for Canva File Upload Functionality

Assumptions Made

Due to the proprietary nature of Canva's exact backend limits, the following assumptions are made for test design (these should be validated against the latest internal product requirements):

- 1. Maximum File Size: Images/Documents = 25 MB; Videos/Audio = 250 MB (standard Canva limits).**
- 2. Minimum File Size: 1 KB (0-byte files are rejected).**

3. **Resume Capability:** Interrupted uploads > 10 MB support resumption.
4. **Cloud Storage:** Google Drive and Dropbox integrations are active in the upload panel.
5. **Concurrent Upload Limit:** Maximum of 10 simultaneous file uploads per session.

1. Supported File Types

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
SUP-001	File Upload	Verify upload of supported Image formats (JPG, PNG, SVG, GIF, WEBP, BMP, TIFF)	Positive	User is logged in; Media library is empty.	1. Navigate to Uploads panel. 2. Click "Upload files". 3. Select one valid file of each supported image format sequentially.	image.jpg, image.png, image.svg, image.gif, image.webp, image.bmp, image.tiff (each < 25MB)	Each file uploads successfully, displays a success toast, and appears correctly in the media library with accurate thumbnails.	High
SUP-002	File Upload	Verify upload of supported Document format (PDF)	Positive	User is logged in.	1. Navigate to Uploads panel. 2. Upload a standard PDF document.	document.pdf (10 MB)	Upload completes successfully. PDF appears in the "Projects" or "Docs" section of the media library with a PDF icon preview.	High

SUP-003	File Upload	Verify upload of supported Video formats (MP4, MOV, AVI, WEBM)	Positive	User is logged in.	1. Navigate to Uploads panel. 2. Upload one valid video file of each supported format.	video.mp4, video.mov, video.avi, video.webm (each < 250MB)	Upload completes. Video appears in the media library with a playable preview thumbnail and correct duration metadata.	High
SUP-004	File Upload	Verify upload of supported Audio formats (MP3, WAV)	Positive	User is logged in.	1. Navigate to Uploads panel. 2. Upload valid MP3 and WAV files.	audio.mp3, audio.wav (each < 250MB)	Upload completes. Audio files appear in the media library with an audio waveform icon and correct duration metadata.	High

2. Unsupported File Types

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
---------	--------	---------------	-----------	----------------	------------	-----------	-----------------	----------

UNS-001	File Upload	Verify rejection of unsupported document formats (e.g., DOCX, TXT, XLSX)	Negative	User is logged in.	1. Navigate to Uploads panel. 2. Attempt to upload a .docx and .txt file via file browser.	document.docx , notes.txt	Upload is blocked or fails immediately. A clear, user-friendly error message is displayed: "File type not supported. Please upload JPG, PNG, PDF, MP4, etc."	High
UNS-002	File Upload	Verify rejection of unsupported archive formats (e.g., RAR, 7Z)	Negative	User is logged in.	1. Navigate to Uploads panel. 2. Attempt to upload a .rar file.	archive.rar	Upload is rejected. Error message indicates the format is not supported. File does not appear in the media library.	Medium

3. File Size Validation

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
SIZ-001	File Upload	Verify rejection of zero-byte files	Boundary	User is logged in.	1. Create a 0-byte file. 2. Attempt to upload it via the file browser.	empty.png (0 KB)	Upload is rejected immediately. Error message: "File is empty or corrupted."	High

SIZ-002	File Upload	Verify upload of file exactly at the maximum size limit (25 MB)	Boundary	User is logged in.	1. Generate a valid PNG file exactly 25.00 MB. 2. Attempt upload.	exact_limit.png (25.00 MB)	Upload completes successfully. File appears in the media library.	High
SIZ-003	File Upload	Verify rejection of file just above the maximum size limit	Boundary	User is logged in.	1. Generate a valid PNG file of 25.01 MB. 2. Attempt upload.	over_limit.png (25.01 MB)	Upload is rejected. Error message: "File size exceeds the 25 MB limit."	High
SIZ-004	File Upload	Verify handling of extremely large files (e.g., 2 GB)	Edge	User is logged in.	1. Attempt to upload a 2 GB video file.	huge_video.mp4 (2 GB)	Client-side validation intercepts the upload immediately, preventing unnecessary network traffic. Clear error message displayed.	Medium

SIZ-005	File Upload	Verify combined size validation for multiple file uploads	Boundary	User is logged in.	1. Select 15 files, each 20 MB (Total 300 MB). 2. Attempt simultaneous upload.	15 x image.png (20 MB each)	System either queues them respecting a session limit or rejects the batch with a message: "Total upload size exceeds session limits."	Medium
---------	-------------	---	----------	--------------------	---	-----------------------------	---	--------

4. Upload Operations

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
OPR-001	File Upload	Verify drag-and-drop upload functionality	Positive	User is logged in; Upload panel is open.	1. Open OS file explorer. 2. Drag a valid image file and drop it into the Canva upload dropzone.	drag_image.jpg (5 MB)	Dropzone highlights during drag. File uploads successfully upon drop, showing a progress bar and success state.	High

OPR-002	File Upload	Verify cancelation of an ongoing upload	Positive	User is logged in; uploading a large file.	1. Start uploading a 100 MB video. 2. Click the "Cancel" (X) button on the progress indicator mid-upload.	large_video.mp4 (100 MB)	Upload halts immediately. Progress bar disappears. No partial file is saved to the media library.	High
OPR-003	File Upload	Verify retry mechanism for a failed upload	Positive	User is logged in.	1. Simulate a temporary network drop during upload. 2. Click the "Retry" button that appears on the failed upload card.	retry_image.png (10 MB)	Upload resumes or restarts successfully. Progress bar updates, and file appears in the library upon completion.	High
OPR-004	File Upload	Verify upload progress indicator accuracy	Positive	User is logged in.	1. Upload a 50 MB file on a throttled network. 2. Observe the progress bar and percentage.	progress_test.mp4 (50 MB)	Progress bar smoothly increments from 0% to 100%. Time remaining estimate is reasonably accurate.	Medium

OPR-005	File Upload	Verify simultaneous upload of multiple files	Positive	User is logged in.	1. Select 5 different valid files. 2. Click Upload.	5 mixed valid files (< 25 MB each)	5 distinct progress indicators appear. All files upload concurrently and appear in the media library upon completion.	High
---------	-------------	--	----------	--------------------	--	------------------------------------	---	------

5. Success Scenarios

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
SUC-001	File Upload	Verify accurate metadata display post-upload	Positive	User is logged in.	1. Upload a valid file. 2. Hover over or click "Info" on the uploaded asset in the media library.	metadata_test.jpg (Name: "My Design", Size: 2.4 MB, Type: JPEG)	Metadata panel accurately displays the exact file name, size (rounded appropriately), type, and upload date.	High
SUC-002	File Upload	Verify uploaded asset can be inserted into a design	Positive	User is logged in; a blank design is open.	1. Upload a valid image. 2. Drag the uploaded image from the media library onto the design canvas.	canvas_test.png (5 MB)	Image snaps to the canvas, renders correctly without distortion, and is selectable/editable.	High

SUC-003	File Upload	Verify success message and UI feedback	Positive	User is logged in.	1. Complete a successful single file upload.	success_test.pdf (3 MB)	A non-intrusive success toast notification appears (e.g., "Upload complete"). The file is immediately visible in the "Recent uploads" section.	Medium
---------	-------------	--	----------	--------------------	--	-------------------------	--	--------

6. Failure Scenarios

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
FAIL-001	File Upload	Verify behavior when network connection is lost mid-upload	Negative	User is logged in; uploading a large file.	1. Start uploading a 50 MB file. 2. Disconnect Wi-Fi/Ethernet mid-upload.	network_loss.mp4 (50 MB)	Upload pauses/fails. UI displays "Connection lost. Retrying..." or a "Retry" button. No corrupted partial file is shown in the library.	High
FAIL-002	File Upload	Verify handling of server-side 500 Internal Server Error	Negative	User is logged in.	1. Mock a 500 error response from the upload endpoint. 2. Attempt to upload a valid file.	server_error.png (5 MB)	Upload fails gracefully. User sees a friendly message: "Something went wrong on our end. Please try again later." No stack traces exposed.	High

FAIL-003	File Upload	Verify behavior when user storage quota is exceeded	Negative	User is logged in; storage quota is at 99.9%.	1. Attempt to upload a 10 MB file.	quota_exceed.jpg (10 MB)	Upload is blocked. Clear message: "You have reached your storage limit. Please upgrade or delete files to continue."	High
FAIL-004	File Upload	Verify handling of corrupted image files	Negative	User is logged in.	1. Take a valid JPG and corrupt its header bytes. 2. Attempt upload.	corrupted.jpg (5 MB)	Server/client validation detects corruption. Upload fails with message: "File is corrupted or unreadable."	Medium

7. Security Testing

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
SEC-001	File Upload	Verify rejection of malicious executable files	Security	User is logged in.	1. Rename a harmless .txt file to .exe, .bat, .cmd, or .sh. 2. Attempt upload.	malware.exe, script.sh	Upload is strictly blocked by MIME/extension validation. Error: "File type not allowed."	Critical

SE C-02	File Upload	Verify protection against double-extension attacks	Security	User is logged in.	1. Create a file named image.jpg.exe. 2. Attempt upload.	image.jpg.exe	System rejects the file based on the final .exe extension or MIME type mismatch.	Critical
SE C-03	File Upload	Verify prevention of XSS via file name	Security	User is logged in.	1. Name a valid image file: <script>alert('XSS')</script>.png. 2. Upload and view in media library.	<script>alert(1)</script>.png	File name is sanitized/escaped in the UI. The script does not execute. Displayed as literal text or hashed.	Critical
SE C-04	File Upload	Verify prevention of SQL Injection via file name	Security	User is logged in.	1. Name a file: image'; DROP TABLE users;--.png. 2. Attempt upload.	image'; DROP TABLE users;--.png	Upload succeeds (if valid image), but the filename is safely parameterized/sanitized in the database. No DB errors or data loss.	Critical
SE C-05	File Upload	Verify defense against Directory Traversal in file names	Security	User is logged in.	1. Name a file: ../../../../etc/passwd.png. 2. Attempt upload.	../../../../etc/passwd.png	System strips or rejects path traversal characters. File is saved safely in the user's designated sandboxed directory with a sanitized name.	Critical
SE C-06	File Upload	Verify MIME type validation (preventing disguised files)	Security	User is logged in.	1. Rename a .html file containing JS to fake_image.jpg.	fake_image.jpg (actual content: HTML/JS)	Server-side MIME inspection detects mismatch between extension and actual content.	Critical

					2. Attempt upload.		Upload is rejected.	
SEC-007	File Upload	Verify handling of ZIP bombs (Decompression bomb)	Security	User is logged in.	1. Attempt to upload a highly compressed ZIP file (e.g., 42.zip) if ZIP uploads were ever supported, or disguised as allowed type.	zipbomb.zip	System detects abnormal compression ratio or blocks archive formats entirely. Resource exhaustion is prevented.	High
SEC-008	File Upload	Verify unauthorized upload attempts (Broken Access Control)	Security	User is logged out or using a different user's session token.	1. Intercept a valid upload request. 2. Replay it with an invalid or different user's session token.	Valid payload, Invalid Auth	Server returns 401 Unauthorized or 403 Forbidden. File is not uploaded.	Critical

8. Performance Testing

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
PERF-001	File Upload	Verify upload performance under slow network (3G throttling)	Performance	User is logged in; Network throttled to 3G (e.g., 1.5 Mbps).	1. Upload a 10 MB image. 2. Monitor time and UI responsiveness.	slow_net.jpg (10 MB)	Upload completes within acceptable time (e.g., < 2 mins). UI remains responsive; progress bar updates smoothly without freezing	High

							the browser.	
PER F-002	File Upload	Verify memory and CPU utilization during large batch upload	Performance	User is logged in; DevTools open.	1. Select 50 valid images (total 100 MB). 2. Upload simultaneously. 3. Monitor Memory/CPU tabs.	50 x batch_image.jpg	Memory usage remains stable (no memory leaks). CPU spikes are brief and normalize. Browser does not crash or become unresponsive.	High
PER F-003	File Upload	Verify recovery after application/browser crash mid-upload	Performance	User is logged in; upload in progress.	1. Start uploading a mp4 (100 MB file). 2. Force-close the browser tab. 3. Reopen Canva and navigate to media library.	crash_test.mp4 (100 MB)	No corrupted "ghost" files appear in the library. If resume is supported, a "Resume upload" prompt may appear upon returning to the upload panel.	Medium

PERF-004	File Upload	Verify mobile background/foreground upload behavior	Performance	User is logged in on mobile device (iOS/Android).	1. Start uploading a 20 MB video. 2. Switch to another app (background the browser). 3. Return to Canva after 1 minute.	mobile_bg.mp4 (20 MB)	Upload either continues seamlessly in the background (if PWA/native app supports it) or pauses gracefully and prompts to resume upon returning to foreground.	Medium
----------	-------------	---	-------------	---	---	-----------------------	---	--------

9. Compatibility Testing

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
COMP-001	File Upload	Verify upload functionality on Chrome (Windows)	Compatibility	User logged in on Windows 11, Chrome (latest).	1. Perform drag-and-drop and file browser upload.	test_chrome.png	Upload functions flawlessly. UI elements render correctly.	High
COMP-002	File Upload	Verify upload functionality on Safari (macOS)	Compatibility	User logged in on macOS Sonoma, Safari (latest).	1. Perform file browser upload.	test_safari.jpg	Upload functions flawlessly. No WebKit-specific file picker or MIME issues.	High

COM P-003	File Upload	Verify upload functionality on Mobile Safari (iOS)	Compatibility	User logged in on iOS 17, Safari.	1. Tap "Upload files". 2. Select "Photo Library" and choose an image/video.	test_ios.heic or .jpg	OS file picker opens correctly. HEIC files are either auto-converted or accepted. Upload succeeds.	High
COM P-004	File Upload	Verify upload functionality on Chrome (Android)	Compatibility	User logged in on Android 14, Chrome.	1. Tap "Upload files". 2. Select from device storage.	test_android.mp4	File picker functions correctly. Upload progress is visible and completes successfully.	High
COM P-005	File Upload	Verify upload via Cloud Storage integration (Google Drive)	Compatibility	User logged in; Google Drive connected.	1. Click "More" -> "Google Drive". 2. Select and import a valid PDF.	drive_doc.pdf	OAuth flow completes securely. File is fetched and appears in Canva media library without downloading to local device first.	Medium

10. Edge Cases

Test ID	Module	Test Scenario	Test Type	Pre-Conditions	Test Steps	Test Data	Expected Result	Priority
---------	--------	---------------	-----------	----------------	------------	-----------	-----------------	----------

EDGE-001	File Upload	Verify handling of extremely long file names	Edge	User is logged in.	1. Rename a file to 255 characters long. 2. Attempt upload.	[255-char-string].png	System truncates or safely hashes the filename for storage. UI displays a truncated version with an ellipsis (e.g., verylongna...png). Upload succeeds.	Medium
EDGE-002	File Upload	Verify handling of special and Unicode characters in file names	Edge	User is logged in.	1. Name a file with emojis and Unicode: 设计 计 设计_test!.jpg. 2. Attempt upload.	设计 设计_test!.jpg	File uploads successfully. Filename is properly URL-encoded/escaped in transit and displayed correctly in the UI without breaking the layout.	Medium
EDGE-003	File Upload	Verify handling of password-protected PDF files	Edge	User is logged in.	1. Attempt to upload a PDF encrypted with a password.	protected.pdf	Upload may succeed, but preview generation fails gracefully with a message: "Cannot preview password-protected PDF." File remains in library.	Low

EDGE-004	File Upload	Verify duplicate file upload handling	Edge	User is logged in; file already exists in library.	1. Upload duplicate.png. 2. Immediately upload the exact same duplicate.png again.	duplicate.png (identical hash)	System detects duplicate via hash/name. Either blocks the second upload with a message "File already exists" or creates a distinct copy (e.g., duplicate(1).png) based on product rules.	Medium
EDGE-005	File Upload	Verify upload during daylight saving time (DST) transition	Edge	User is logged in.	1. Upload a file exactly during a DST clock shift.	dst_test.jpg	Metadata timestamp is recorded accurately in UTC and displayed correctly in the user's local timezone without a 1-hour offset error.	Low

Review & Sign-off Notes (Persona Perspectives)

- **Senior Test Engineer:** Test cases cover the full lifecycle from client-side validation to server-side processing, ensuring both manual and automation teams can easily script these (e.g., using Playwright/Cypress for UI, Postman/REST Assured for API mocking).
- **Security Test Engineer:** Critical vulnerabilities (XSS, SQLi, MIME spoofing, ZIP bombs, path traversal) are explicitly targeted with clear expected mitigation behaviors.
- **Performance Test Engineer:** Scenarios include throttling, resource monitoring, and mobile background states, ensuring the app remains resilient under real-world constraints.
- **Product Owner:** Business rules (file type limits, storage quotas, cloud integrations) are validated to ensure they align with user expectations and tier limitations.

- **UX Specialist:** Emphasis is placed on clear, non-technical error messages, smooth progress indicators, and graceful degradation (e.g., during network loss), ensuring a frictionless user experience.